

PEP 3333 — WSGI Key Points

Python Web Server Gateway Interface v1.0.1

Date: 2026-08-02
Source: <https://peps.python.org/pep-3333/>
PEP Status: Final (Accepted)
Author: Phillip J. Eby
Replaces: PEP 333 (WSGI 1.0)
Created: 26-Sep-2010

WSGI specifies a standard interface between web servers and Python web applications or frameworks, enabling portability across a variety of web servers. It decouples framework choice from server choice and enables composable middleware stacks.

This document extracts key specification points from PEP 3333 for quick reference. Refer to the official PEP at <https://peps.python.org/pep-3333/> for the authoritative text.

Overview & Purpose

- PEP 3333 defines WSGI v1.0.1 — the standard interface between web servers and Python web applications/frameworks.
- It is an update to PEP 333 (WSGI 1.0), adding Python 3 compatibility and codifying de-facto amendments.
- A PEP 333-compliant Python 2.x server or application is also PEP 3333-compliant.
- Goal: decouple web framework choice from web server choice, similar to Java's Servlet API.
- WSGI is a tool for framework and server developers — not a direct API for application developers.
- WSGI enables 'middleware stacks': loosely-coupled components that are simultaneously servers and applications.
- The specification requires no new standard library modules and targets Python $\geq 2.2.2$.

String Types (Critical for Python 3)

- Two distinct string kinds exist in WSGI:
 - 'Native' strings (type `str`): used for request/response headers and metadata.
 - 'Bytestrings' (type `bytes` in Py3, `str` in Py2): used for request/response bodies.
- Even in Python 3 where `str` is Unicode, native strings must only contain Latin-1 (ISO-8859-1) code points (U+0000–U+00FF).
- Header names/values passed to `start_response()` must be `str` (native), never bytes.
- Body data yielded by the application and passed to `write()` must be bytes in Python 3.
- Mixing `str` and bytes where the other is required is a fatal protocol violation.

Application / Framework Side

- An application object is any callable accepting exactly two positional arguments: `environ` and `start_response`.
- Valid callable forms: plain function, method, class (instantiated per request), or instance with `__call__`.
- Invocation: `result = application(environ, start_response)` — positional args only, never keyword.
- Application must be re-entrant: servers (except pure CGI) will call it multiple times.
- Application must call `start_response()` before yielding the first non-empty body bytestring.
- Application must return an iterable yielding zero or more bytestrings (the response body).
- If the returned iterable has a `close()` method, the server MUST call it after the request completes (even on error).
- Applications returning a generator should not assume the entire iterator will be consumed.
- WSGI is intentionally 'no frills' — no response objects, no cookie helpers; those belong in the framework layer.

Server / Gateway Side

- The server invokes the application callable once per HTTP request directed at the application.
- The server builds the environ dict (CGI variables + WSGI-specific variables) and calls the application.
- The server must transmit each yielded bytestring to the client in an unbuffered fashion before requesting the next.
- The server must call `iterable.close()` upon request completion, whether normal or on error.
- If the application provides a Content-Length header, the server must NOT send more bytes than specified.
- If Content-Length is absent, the server may: close the connection, auto-calculate (single-item iterable), or use HTTP/1.1 chunked Transfer-Encoding.
- The server must handle inbound 'hop-by-hop' headers (e.g., Transfer-Encoding: chunked) on its own — the application never sees them raw.
- Servers SHOULD provide a single-threaded mode so non-thread-safe applications can still run.
- HTTP/1.1 servers must transparently handle Expect: 100-continue (respond with 100, lazy-send, or wait for client timeout).

environ Variables — CGI Variables

- The environ parameter must be a plain Python dict (not a subclass or UserDict).
- The application may modify the dict freely.
- Required CGI variables (omit only if value would be empty string, except where noted):
 - REQUEST_METHOD — HTTP method string, e.g. 'GET', 'POST'. Never empty; always required.
 - SCRIPT_NAME — URL path prefix mapping to the application root. May be empty string.
 - PATH_INFO — Remainder of the URL path beyond SCRIPT_NAME. May be empty string.
 - QUERY_STRING — Everything after '?' in the URL. May be empty or absent.
 - CONTENT_TYPE — Value of the Content-Type request header. May be empty or absent.
 - CONTENT_LENGTH — Value of the Content-Length request header. May be empty or absent.
 - SERVER_NAME — Server hostname; combined with SERVER_PORT when HTTP_HOST absent. Never empty.
 - SERVER_PORT — Server port as a string, e.g. '80', '443'. Never empty.
 - SERVER_PROTOCOL — Protocol version, e.g. 'HTTP/1.0' or 'HTTP/1.1'.
 - HTTP_* — Client-supplied HTTP headers, prefixed with HTTP_. E.g. HTTP_HOST, HTTP_ACCEPT.
- All CGI variables must be native str type; any other type is a specification violation.
- Missing variables (e.g. REMOTE_USER when unauthenticated) must be omitted entirely, not set to empty.

environ Variables — WSGI-Specific Variables

- `wsgi.version` — Tuple (1, 0) indicating WSGI version 1.0.
- `wsgi.url_scheme` — String 'http' or 'https' representing the URL scheme.
- `wsgi.input` — File-like binary input stream for reading the HTTP request body.
 - Supports: `read(size)`, `readline([size])`, `readlines([hint])`, `__iter__()`.
 - Server must simulate EOF if app reads past `CONTENT_LENGTH`.
- `wsgi.errors` — Text-mode output stream for error logging (e.g. `sys.stderr` or server log).
 - Supports: `flush()`, `write(str)`, `writelines(seq)`.
 - Applications must use `'\\n'` as line ending; server converts as needed.
- `wsgi.multithread` — True if the app object may be invoked concurrently by another thread.
- `wsgi.multiprocess` — True if an equivalent app object may be invoked in another process.
- `wsgi.run_once` — True if the server expects this to be the app's only invocation (e.g., pure CGI).
- Server-specific extension variables: lowercase names with unique server prefix, e.g. `mod_python.some_var`.

The `start_response()` Callable

- Signature: `start_response(status, response_headers, exc_info=None)`
- Must be called with positional arguments only.
- `status`: a native string of the form '999 Message', e.g. '200 OK', '404 Not Found'.
 - Must not contain control characters, carriage returns, or linefeeds.
- `response_headers`: a Python list of (header_name, header_value) tuples.
 - `header_name`: valid HTTP field name, no trailing colon.
 - `header_value`: no control characters including CR/LF.
 - Server is responsible for adding required headers (Date, Server) if app omits them.
 - Apps and middleware MUST NOT use hop-by-hop headers (Connection, Keep-Alive, Transfer-Encoding, etc.).
- `start_response()` must NOT immediately transmit headers — it stores them for later transmission.
- Headers are sent only upon the first non-empty body bytestring, or when `write()` is called.
- Exception: headers with Content-Length: 0 may be sent immediately.
- Returns the `write()` callable.
- `exc_info` parameter: supply only when handling an error. Must be a `sys.exc_info()` tuple.
 - If headers not yet sent: replaces stored headers and returns normally.
 - If headers already sent: MUST re-raise the original exception.
 - Implementers must set `exc_info = None` in a finally block to avoid circular references.
- Calling `start_response()` a second time without `exc_info` is a fatal error.

The `write()` Callable

- Returned by `start_response()`; signature: `write(body_data)` where `body_data` is a bytestring.
- Exists only for backward compatibility with imperative-output frameworks.
- New applications and frameworks SHOULD NOT use `write()` if avoidable.
- `write()` must guarantee the bytestring is fully sent or buffered for transmission before returning.

PEP 3333 — WSGI Key Points

- Calling `write()` before `start_response()` is an error (headers not yet set).
- An application that uses `write()` must still return an iterable (may be empty).
- Bytes yielded by the iterable are sent AFTER all bytes passed to `write()` have been transmitted.
- Middleware must NOT use the parent server's `write()` to transmit data yielded by a wrapped application.

Buffering and Streaming

- Best practice: buffer all output and return a single-element list — best throughput for typical HTML responses.
- For large files or server-push (multipart), return an iterator/generator producing blocks.
- Servers/gateways MUST NOT delay block transmission; each block must be sent or queued before asking for the next.
- Three valid strategies for guaranteeing non-delayed delivery:
 - 1. Fully transmit the block to the OS (flush O/S buffers) before returning control.
 - 2. Use a separate thread to continue transmitting while the app produces the next block.
 - 3. (Middleware only) pass the entire block up to the parent gateway/server.
- Middleware block boundaries: middleware MUST yield at least one value for every value the wrapped app yields.
- If middleware needs more data before producing output, it must yield an empty bytestring `b''` in the meantime.
- Middleware must NOT block waiting to accumulate multiple values before yielding.
- Applications should NOT apply Transfer-Encoding (chunking, gzip) — that is the server's responsibility.

Middleware

- Middleware is simultaneously an application (to its containing server) and a server (to a wrapped app).
- Middleware is transparent to both sides; neither server nor application requires special support to use it.
- A 'middleware stack' is a chain of middleware wrapping one another and ultimately wrapping the real application.
- Common middleware uses:
 - URL routing: dispatch to different apps based on `PATH_INFO`, rewriting `environ` accordingly.
 - Running multiple apps/frameworks side-by-side in the same process.
 - Load balancing and remote forwarding.
 - Content postprocessing: XSL transforms, compression, response rewriting.
- Middleware must conform to BOTH server-side and application-side requirements of WSGI.
- Middleware must not use `write()` to transmit data yielded by a wrapped application.
- Middleware providing extension data (cookies, sessions) should expose functions on `environ`, not stuff values in.

Error Handling

- Applications should catch their own internal errors and produce an appropriate error response.
- If no body data has been sent yet, call `start_response(status, headers, sys.exc_info())` to replace stored headers.
- If body data has already been sent, `start_response` with `exc_info` MUST re-raise — aborting the response.
- Applications must NOT trap exceptions raised by `start_response()` when `exc_info` is provided.
- Servers should trap and log any exception that aborts an application or its response iteration.
- If a partial response has been sent with a `text/*` content type, the server MAY append an error message.
- Pattern for safe error handling in applications:
 - `try: ... start_response('200 OK', headers); return [body]`
 - `except: start_response('500 Error', err_headers, sys.exc_info()); return [err_body]`

URL Reconstruction

- Full request URL can be reconstructed from environ variables:
 - `scheme = wsgi.url_scheme`
 - `host = HTTP_HOST` (or `SERVER_NAME + ':' + SERVER_PORT` if `HTTP_HOST` absent)
 - `path = SCRIPT_NAME + PATH_INFO` (ensure at least `'/'` if both are empty)
 - `query = '?' + QUERY_STRING` (if `QUERY_STRING` is non-empty)
 - `URL = scheme + '://' + host + path + query`
- Note: the reconstructed URL may not be byte-for-byte identical to the original client URL.

Relationship to PEP 333

- PEP 3333 replaces PEP 333 (WSGI 1.0, originally written for Python 2).
- The core protocol is unchanged — any PEP 333-compliant Python 2 server/app is also PEP 3333-compliant.
- Key additions in PEP 3333:
 - Explicit two-string-type model (native `str` vs. `bytes`) for Python 3 compatibility.
 - Clarified unicode rules: native strings restricted to Latin-1 code points.
 - Codified `readline(size)` support for `wsgi.input` (previously optional/de-facto).
 - Code examples ported to Python 3.
 - De-facto amendments from years of real-world WSGI implementations incorporated.
- PEP 3333 was published 26-Sep-2010, Status: Final (Accepted).
- Author: Phillip J. Eby. Replaces: PEP 333.